

NR2DT-01: Step 1 — Discover

Series: NR2DT | Notebook: 1 of 10 | Created: April 2026 | Last Updated: 04/14/2026

Overview

Goal of this step: produce a complete inventory of the source New Relic account, a translation-confidence report, and a defensible effort estimate. Nothing else moves until discovery is complete.

This is procedural. For the *why* behind each artifact and the deeper translation theory, see **NRLC-02** (NRQL→DQL Translation) and the relevant NRLC component notebook for each entity class.

Table of Contents

- 1. [What You'll Produce](#)
- 2. [Prerequisites & Access](#)
- 3. [Run the Inventory](#)
- 4. [Translate the NRQL Surface](#)
- 5. [Generate the Effort Estimate](#)
- 6. [Step Exit Criteria](#)

Prerequisites

Requirement	Details
Audience	Migration lead + assigned engineer for this step

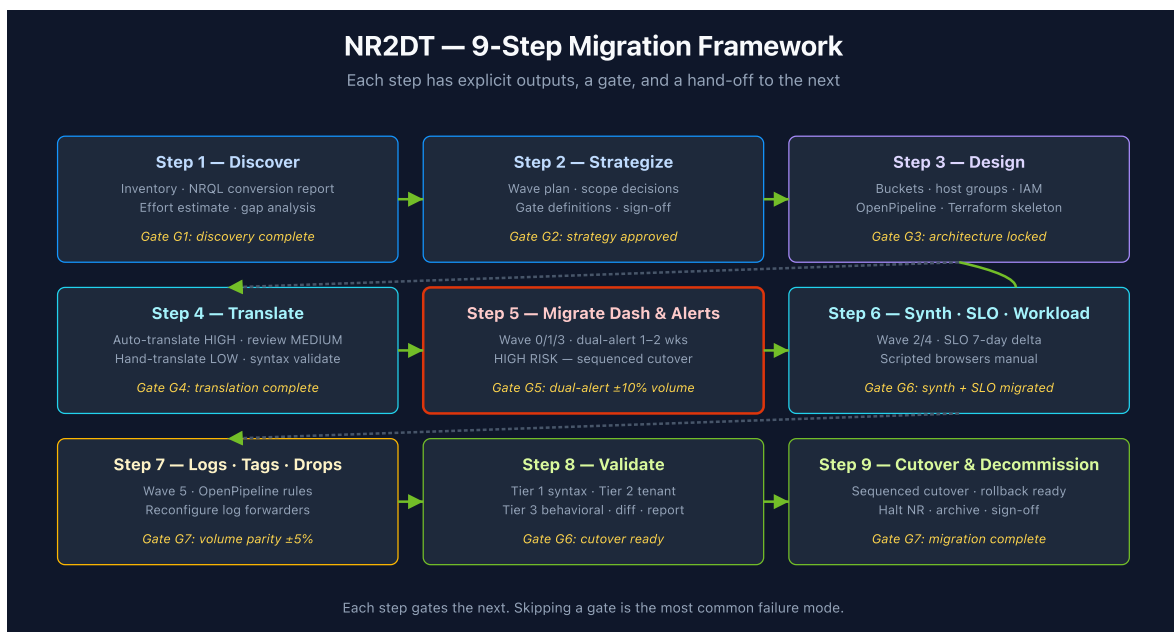
Requirement	Details
Format	Procedural step — use as a runbook; defer to NRLC for depth
NRLC deep dives	NRLC-01 (Platform Comparison) for orientation

1. What You'll Produce

By the end of this step you will have these artifacts checked into your migration repo:

Artifact	Format	Purpose
<code>inventory.json</code>	JSON	Full enumeration of all NR entity classes
<code>nrql-conversion-report.csv</code>	CSV	Every NRQL query with HIGH/MEDIUM/LOW confidence + DQL output
<code>effort-estimate.md</code>	Markdown	Hours/days/weeks per component class
<code>gap-analysis.md</code>	Markdown	Entities flagged for manual work (APM conditions, scripted browsers)
<code>risk-register.md</code>	Markdown	Known risks with proposed mitigations

These artifacts gate Step 2 (Strategize). Don't proceed without them.



2. Prerequisites & Access

NR API token

User-level API key with **read** scope on:

- Dashboards, Alerts, NRQL, Synthetics, SLO, Workloads, Logs

```
export NR_API_KEY="NRAK-..."
export NR_ACCOUNT_ID="&lt;numeric&gt;"
export NR_REGION="US" # or EU
```

Tooling

Clone the migration framework:

```
git clone https://github.com/timstewart-dynatrace/Dynatrace-NewRelic
cd Dynatrace-NewRelic
pip install -r requirements.txt
cp .env.example .env # populate NR_* vars
```

3. Run the Inventory

Command:

```
python3 migrate.py --export-only --output ./inventory
```

This runs the NerdGraph enumeration for every entity class and writes per-component JSON files plus the consolidated `inventory.json`.

Check the output:

```
ls -lh inventory/
head inventory/inventory.json | jq '.summary'
```

Expected counts to capture (your numbers, for the wave plan in Step 2):

- Dashboards (and total widget count across all pages)
- Alert policies + NRQL conditions + APM conditions

- Synthetic monitors by type (Ping / Browser / API / Step)
- SLOs
- Workloads
- Notification channels by type
- Drop rules + log parsing rules + tag rules

4. Translate the NRQL Surface

Every dashboard widget, alert condition, and SLO indicator contains a NRQL query that must compile to DQL. Run the compiler now to size the translation effort.

Command:

```
python3 migrate.py compile --file inventory/all-nrql.txt --report
```

Output: `nrql-conversion-report.csv` with one row per query: source query, translated DQL, confidence (HIGH/MEDIUM/LOW), confidence score (0–100), notes, warnings.

Expected confidence distribution for a typical NR account:

Confidence	Typical share	Action in Step 4
HIGH	70–80%	Auto-translate; sample-validate during dual-run
MEDIUM	15–25%	100% review during dual-run
LOW	0–10%	Hand-translate or reformulate

If LOW is > 15%, surface this to stakeholders early — it's the dominant cost driver.

Deep-dive reference: NRLC-02 explains the compiler architecture, the 292 patterns, and how confidence is scored.

5. Generate the Effort Estimate

Run the report generator to produce a defensible effort estimate:

```
python3 migrate.py --report --input inventory --output report.md
```

Estimating heuristic per component class:

Class	Light (hrs)	Moderate (days)	Heavy (weeks)
Dashboards	< 20	20–80	80+
Alert conditions	< 50	50–200	200+
Synthetics (incl. scripted)	< 10	10–40	40+
SLOs	< 5	5–20	20+
Notification channels (with secrets)	< 5	5–20	20+

6. Step Exit Criteria

G1 — Discovery Complete

Pass criteria — all five must be true:

- [] `inventory.json` complete (no enumeration errors in log)
- [] Per-component counts captured and documented
- [] `nrql-conversion-report.csv` generated; LOW count $\leq$ 15% of total
- [] `effort-estimate.md` reviewed by lead
- [] `gap-analysis.md` reviewed; manual-work items have estimated owners

If any check fails, do not proceed to Step 2. Fix the gap first.

Next step: NR2DT-02 — Strategize (wave planning, scope decisions, gate definitions for the rest of the migration).

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source [Dynatrace-NewRelic](#), [nrql-engine](#), and [nrql-translator](#) projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official [Dynatrace documentation](#) and [New Relic documentation](#).